

Hardware/Software Co-Design for Image Processing Based on RISC-V Architecture

Embedded Foreground Detection and M2 Accelerator Integration

Report Type: Final report

Scope: Algorithm, Interface Contract, Deployment, and Validation

Date: April 29, 2026

Context: This report summarizes the accomplishments and engineering workflows developed during the Spring 2026 Independent Study.

Executive Summary

This report presents an engineering-oriented review of an embedded foreground detection system based on software-hardware co-design. It covers algorithm selection, board-side deployment constraints, streaming-based observability, interface-contract design between software BGS and custom hardware M2, and a reproducible gateway build and flashing workflow. The central principle is to keep interface contracts stable while enabling incremental performance optimization and long-term evolution.

Summary of Achievements

- 1) Keep hardware focused on fact computation and software focused on policy interpretation.
- 2) Treat control block and memory layout as strict contract invariants.
- 3) Validate with a small hand-computable loop before large-scale optimization.
- 4) Use scripted gateway build and rollback-ready flashing as default engineering discipline.

Contents

1	Introduction	3
2	Algorithm Selection and Engineering Adaptation	3
3	Cross-Compilation and Board Deployment	3
4	GStreamer-Based Remote Visualization	4
4.1	Network Configuration and Pre-requisites	4
4.2	Link Validation (Connectivity Test)	4
4.2.1	Sender (Board Side)	4
4.2.2	Receiver (PC Side)	4
4.3	Prototype Deployment: Receiving Application Stream	5
5	Software Baseline: The ViBe Algorithm (C3R Logic)	5
5.1	Segmentation Logic: Unified Color Distance	5
5.1.1	Distance Metric	6
5.1.2	Voting Mechanism	6
5.2	Update Logic: Synchronous Tri-Channel Injection	6
6	Original Algorithm Performance Baseline	7
6.1	Input Video Specifications	7
6.2	Software-only Execution Metrics	8
7	Hardware Architecture: The M2 Co-Processor Design	8
7.1	Optimized Interleaved Memory Layout	8
7.2	Massive Parallel Computation vs. Software Heuristics	9

8	ViBe Adaptation for M2 Co-Execution	10
8.1	Data Structure Reorganization: The Shared Memory Contract	10
8.2	Hardware-Friendly Pixel Format: RGB to BGRX32	10
8.3	The Four-Stage Co-Execution Flow	10
8.4	Hybrid Execution: MMIO vs. Simulator Paths	11
8.5	Result Interpretation and Background Update	11
9	Control Block and Shared-Memory Contract	11
9.1	Register Map and APB Interface	12
9.2	Shared-Memory Layout and Alignment	12
9.3	Execution Flow and Invariants	12
10	M2 Hardware Architecture and Matching Logic	13
10.1	System Interconnect: Dual-Plane Architecture	13
10.2	Data Alignment and AXI Bandwidth Optimization	13
10.3	Fixed-Point Computation and Pre-calculated Thresholds	14
10.3.1	SAD Matching Rule	14
10.3.2	Pre-calculation Strategy	14
10.4	Evidence Word Generation	14
11	State Machine, CDC, and Error Policy	14
12	Performance and Validation Methodology	15
13	Gateway Build and Flashing Workflow	15
14	Evidence from Source Structure and Commit History	16
15	System Collaboration and Hardware Logic	17
15.1	System-Level Collaboration Flow	17
15.2	Hardware Finite State Machine (FSM) Abstraction	17
16	Conclusion	18
17	Summary and Semester Reflection	18

1 Introduction

This review focuses on foreground detection in embedded scenarios. The goal is not merely to execute a background subtraction algorithm, but to build a software-hardware co-design pipeline on a resource-constrained platform that is runnable, observable, maintainable, and evolvable. The architecture follows a layered design: the software side handles task organization, policy scheduling, and result interpretation, while the hardware side handles high-frequency, repetitive, and data-intensive pixel comparison. This boundary provides acceleration benefits without sacrificing policy iteration flexibility.

2 Algorithm Selection and Engineering Adaptation

During algorithm selection, the project evaluated options using the open-source background modeling library `bgslibrary` (<https://github.com/andrewssobral/bgslibrary>). After comparing detection behavior, implementation complexity, and deployment cost, Vibe was selected as the main algorithm. The key reason is that Vibe is lightweight, structurally regular, and relatively controllable in parameters, providing a practical balance between real-time performance and robustness in embedded environments.

After selecting Vibe, I performed simplification of the open-source implementation. The core modeling and foreground decision path was retained, while modules irrelevant to the target platform or replaceable by simpler components were removed.

3 Cross-Compilation and Board Deployment

A major constraint is limited storage on the development board, which cannot host a full generic OpenCV distribution. Therefore, the project adopted an on-demand cross-compilation strategy:

1. Prepare OpenCV source code and a RISC-V toolchain in a WSL environment on an x86 host.
2. Configure compiler, target architecture, and library paths via `riscv-toolchain.cmake`.
3. Compile only required modules.
4. Deploy outputs under the board-side `usr` directory.

This approach avoids full-package size overhead and reduces maintenance redundancy.

4 GStreamer-Based Remote Visualization

Due to the absence of a local display interface on the development board, the visualization path was refactored from a traditional GUI output to a network-based streaming solution. This approach utilizes `textbfGStreamer` to encapsulate the processed video into `textbfRT-P/UDP` packets, which are then transmitted to a host machine for real-time decoding and display.

4.1 Network Configuration and Pre-requisites

During the integration phase, a dedicated point-to-point network was established with the following static parameters:

- **Windows Host IP:** 192.168.7.3 (Receiver)
- **Linux Board IP:** 192.168.7.2 (Sender)
- **Target Port:** 5000 (Standard UDP streaming port)

On the Windows host, the firewall must be configured to allow inbound UDP traffic on the specified port. This can be achieved via PowerShell with administrative privileges:

Listing 1: Windows Firewall Exception

```
1 New-NetFirewallRule -DisplayName "GStreamer_UDP_In" -Direction Inbound -  
   Action Allow -Protocol UDP -LocalPort 5000
```

4.2 Link Validation (Connectivity Test)

To verify the integrity of the network link without the complexity of the ViBe application, a standard test source (`videotestsrc`) is used.

4.2.1 Sender (Board Side)

The board generates a SMPTE test pattern (color bars) and encodes it using H.264 with zero-latency tuning:

```
1 gst-launch-1.0 videotestsrc ! x264enc tune=zerolatency ! rtph264pay ! udpsink  
   host=192.168.7.3 port=5000
```

4.2.2 Receiver (PC Side)

Upon successful connection, the receiver will display a window showing the test pattern, confirming that the UDP pipeline is functional:

```
1 gst-launch-1.0 udpsrc port=5000 ! application/x-rtp ! rtph264depay !
   decodebin ! autovideosink
```

4.3 Prototype Deployment: Receiving Application Stream

Once the link is validated, the receiver is switched to a specialized pipeline designed to handle the actual application stream generated by the ViBe logic. This command explicitly defines the RTP payload parameters to ensure synchronization:

Listing 2: PC-side Final Receiving Command

```
1 gst-launch-1.0 -v udpsrc port=5000 caps="application/x-rtp,media=video,
   encoding-name=H264,payload=96,clock-rate=90000" ! rtph264depay !
   avdec_h264 ! videoconvert ! autovideosink sync=false
```

Key Implementation Details:

- **Caps Definition:** The explicit definition of `encoding-name=H264` and `payload=96` is critical for the receiver to correctly interpret the incoming UDP bitstream without a formal handshake.
- **Sync=false:** This parameter is added to the `autovideosink` to minimize buffering latency, ensuring the visualization reflects the real-time processing state of the FPGA-accelerated ViBe algorithm.

5 Software Baseline: The ViBe Algorithm (C3R Logic)

The system utilizes the **C3R (3-Channel Color)** variant of the ViBe algorithm. The core philosophy of this implementation is to treat the Red, Green, and Blue channels as a **unified vector** (R, G, B) rather than three independent grayscale entities. This ensures that the color structural integrity of the background model is preserved during both the segmentation and update phases.

5.1 Segmentation Logic: Unified Color Distance

In the C3R model, the similarity between a candidate pixel and a background sample is determined by the **Sum of Absolute Differences (SAD)** across all three channels, compared against a scaled empirical threshold.

5.1.1 Distance Metric

The decision function `distance_is_close_8u_C3R` evaluates proximity using the following criterion:

$$\text{Match} = (|R_1 - R_2| + |G_1 - G_2| + |B_1 - B_2|) \leq 4.5 \times \text{Threshold} \quad (1)$$

This approach implies a textbfJoint Color Similarity strategy:

- **Global Contrast:** A significant deviation in a single channel can be compensated for by high similarity in the other two channels.
- **Cumulative Error:** Small, consistent drifts across all three channels may accumulate and trigger a foreground detection, making the system sensitive to subtle chromatic shifts.

5.1.2 Voting Mechanism

The segmentation process employs a competitive voting strategy:

1. The `segmentation_map` for a pixel is initialized to `matchingNumber - 1`.
2. For every background sample that satisfies the SAD distance criterion, the map value is decremented.
3. If the final value remains > 0 (meaning fewer than `min` matches were found), the pixel is classified as **Foreground**; otherwise, it is **Background**.

5.2 Update Logic: Synchronous Tri-Channel Injection

The update mechanism ensures that the background model remains current without being corrupted by moving objects.

- **Conditional Update:** Only pixels classified as `COLOR_BACKGROUND` in the current frame are eligible to update the history model. This prevents "ghosting" or the accidental learning of moving targets into the background.
- **Atomic Sample Replacement:** When a background pixel is selected for an update (based on a stochastic timer), the entire (R, G, B) triplet is written into the history buffer simultaneously.

Listing 3: Synchronous RGB Update Snippet

```
1 // Ensuring color consistency by writing the full triplet
2 historyBuffer[base_idx] = current_r;
```

```
3 historyBuffer[base_idx + 1] = current_g;  
4 historyBuffer[base_idx + 2] = current_b;
```

By storing and updating the channels together, the algorithm prevents "color bleeding" where a background sample might otherwise consist of an R value from one time instance and G/B values from another, which would lead to a physically impossible background representation.



Figure 1: The ViBe Algorithm

6 Original Algorithm Performance Baseline

Before integrating hardware acceleration, a baseline performance test was conducted using the sequential C++ implementation of the ViBe algorithm. This section records the input video specifications and the resulting computational metrics.

6.1 Input Video Specifications

The test sequence consists of a standard surveillance-style video clip with the following parameters:

- **Resolution:** 640×360 pixels
- **Duration:** 12 seconds (Total: 372 frames)

- **Frame Rate:** 30 FPS
- **Loop Delay:** 33.3333 ms

6.2 Software-only Execution Metrics

The sequential algorithm was executed on the CPU, and the timing data (excluding I/O overhead) was collected for each frame. The results indicate that the software-only approach fails to meet the 30 FPS real-time requirement.

Table 1: ViBe Sequential Execution Time and FPS Summary

Metric	Time (sec)	FPS (1/Time)
Mean	0.079998	12.83
Standard Dev.	0.014424	—
Minimum	0.067895	7.81
Median (P50)	0.074612	13.40
90th Percentile	0.105163	14.57
95th Percentile	0.113700	—
99th Percentile	0.123517	—
Maximum	0.128089	14.73
Total Elapsed Time	67.8 seconds	

Performance Analysis: The data shows a mean processing time of approximately **80 ms per frame**, resulting in an average throughput of **12.83 FPS**. Given the target video’s frame rate of 30 FPS (33.33 ms budget per frame), the current software implementation consumes more than double the allowed time per frame. The significant gap between the Maximum (0.128s) and Minimum (0.067s) execution times further suggests that CPU-based processing is subject to jitter, likely due to the conditional branching logic in the software ViBe update and segmentation steps.

7 Hardware Architecture: The M2 Co-Processor Design

The M2 hardware module is a high-throughput matching engine optimized for parallel pixel processing. To overcome the Von Neumann bottleneck associated with frequent memory access in ViBe, the M2 architecture focuses on **Spatio-Temporal Data Locality**.

7.1 Optimized Interleaved Memory Layout

To maximize DMA burst efficiency and ensure continuous data feeding to the pipeline, the M2 co-processor assumes a specific memory organization. Unlike traditional software buffers that store frames in separate planes, M2 utilizes a **textbfSynchronous Interleaved Layout**:

- **Header Padding:** Each processing block starts with a reserved segment for **Result Write-back**. This allows the hardware to store the segmentation masks without needing to switch memory pages or calculate distant offsets.
- **Pixel-History Interleaving:** The current frame pixel (P_{cur}) and its corresponding history samples ($S_1, S_2, \dots, S_N$) are stored contiguously in memory (e.g., $P, S_1, S_2 \dots S_N, P', S'_1 \dots$).
- **Sequential Streaming:** This layout enables the M2 engine to perform a single long-burst read to fetch all necessary data for one pixel's classification in a single clock cycle sequence, minimizing bus arbitration overhead.

7.2 Massive Parallel Computation vs. Software Heuristics

There is a fundamental difference in how M2 and the baseline software handle the classification workload:

- **Software Heuristic:** The sequential C++ implementation often performs "early exit" checks—specifically checking the first two history samples. If they provide strong evidence, it may skip further comparisons to save CPU cycles.
- **Hardware Full-Pipeline Execution:** In the FPGA implementation, the M2 engine is designed for deterministic latency. It calculates the matching evidence for **all** N **history samples** in parallel for every pixel, regardless of the results of the first few samples.

By leveraging the massive logic gates of the FPGA, M2 provides a "brute-force" yet high-speed calculation that ensures a consistent frame rate, providing the software with a complete textbfMatching Evidence Word for every pixel. This ensures that no potential match is missed, offering higher classification robustness than the cycle-saving software shortcuts.

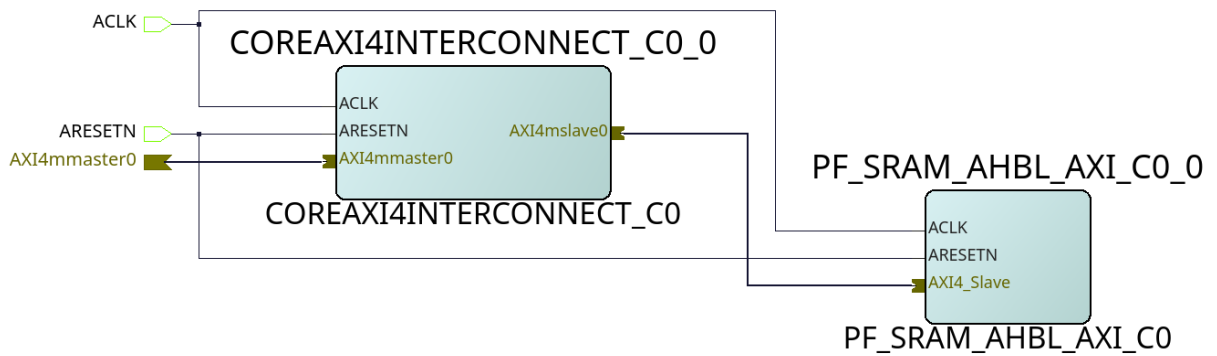


Figure 2: Example Design

8 ViBe Adaptation for M2 Co-Execution

To bridge the gap between the standard ViBe C3R algorithm and the FPGA hardware execution model, the system undergoes a fundamental architectural shift. The adaptation focuses on transforming the algorithm from a "CPU-centric sequential loop" into a "shared-memory-based offloading model."

8.1 Data Structure Reorganization: The Shared Memory Contract

The original software implementation relies on `historyImage` and `historyBuffer` for internal state management. In the M2-adapted version, the `vibeModel` is extended to manage physical hardware interfaces and shared memory regions.

The memory is organized into a **Shared Layout** where offsets for different functional zones are pre-calculated:

- **Frame Zone:** Stores the current BGRX frame to be processed.
- **History Zone:** A contiguous block containing the collection of background samples.
- **Output Zone:** Reserved for the hardware-generated **Matching Evidence Words**.
- **Control Zone:** A memory-mapped control block for synchronization (Start, Done, Status, Error codes).

8.2 Hardware-Friendly Pixel Format: RGB to BGRX32

FPGA architectures favor power-of-two data alignments to simplify address calculation and maximize bus bandwidth. The standard 3-byte RGB format is thus expanded to a **4-byte BGRX32** format.

- **Alignment:** Every pixel is strictly 32-bit aligned, allowing the FPGA to fetch a pixel in a single memory cycle.
- **Transformation:** Software performs a zero-padding write (`dst[3] = 0u`) during the data organization stage. This ensures that the simulated and real FPGA paths share the same data width and memory stride.

8.3 The Four-Stage Co-Execution Flow

The segmentation logic is decoupled into a synchronized four-stage pipeline:

1. **Stage 1 (Preparation):** Software clears the output buffer and initializes the `segmentation_map`.

2. **Stage 2 (Data Marshalling):** The current frame is converted to BGRX and written into the shared `frameSlot`.
3. **Stage 3 (Hardware Trigger):** Software constructs the `Control Block` and triggers execution via MMIO (writing to APB registers) or the simulator path.
4. **Stage 4 (Interpretation):** Software reads the 32-bit `comparisonWord` (evidence bitmap) from the output zone. It calls `fpga::IsForeground(word, matchingNumber)` to derive the final mask based on the bit-count.

8.4 Hybrid Execution: MMIO vs. Simulator Paths

To ensure developability without constant access to physical hardware, the system implements a `textbfDual-Transport Architecture`:

- **MMIO Path:** Utilizes `/dev/mem` to map the FPGA's APB registers and Shared Memory into the user-space, enabling direct hardware control on the BeagleV-Fire.
- **Simulation Path (`vibe-fpga-sim`):** A bit-accurate software model that replicates the M2 engine's behavior. It reads from the shared memory layout, performs SAD-based BGRX comparisons, and updates the control block's "Done" and "Status" bits, mimicking hardware state machine transitions.

8.5 Result Interpretation and Background Update

While the heavy comparison task is offloaded to the M2 engine, the `textbfUpdate Logic` remains in the software domain to maintain flexibility.

- **Evidence-Based Decision:** The FPGA provides the "evidence" (which samples matched), but the software maintains the "policy" (how many matches define background).
- **Shared Update:** When the software performs a stochastic update, it writes the new RGB triplet directly into the **FPGA Shared History Zone**, ensuring the hardware always has the most recent "knowledge" for the next frame's comparison.

9 Control Block and Shared-Memory Contract

The control block serves as the fundamental synchronization and configuration interface between the software host and the hardware accelerator. It is designed as an executable transaction descriptor that encapsulates the operational state and algorithmic parameters of the ViBe (Visual Background Extractor) acceleration logic.

9.1 Register Map and APB Interface

The hardware-software interface is realized through a set of memory-mapped registers via the Advanced Peripheral Bus (APB). To ensure binary compatibility between the C-header (*vibe - fpga - shared.h*) and the RTL implementation (*M2_AXI_ACCEL.v*), the register offsets are strictly defined as follows:

Table 2: Hardware Register Map (APB Interface)

Offset	Register Name	Description
0x00	CONTROL	Handshake bits (Start, Done, Reset, Interrupt Enable).
0x04	CONTROL_BLOCK_ADDR	Physical base address of the Shared Control Block in DRAM.
0x08	DEBUG_PIXEL_INDEX	Target pixel index for hardware-level debugging.
0x0C	DEBUG_HISTORY_INDEX	Target history sample index for sample-level inspection.
0x10	DEBUG_COMPARE_WORD	Expected value for hardware comparison logic testing.
0x14	DEBUG_ERROR_CODE	Status code returned by hardware upon transaction failure.

9.2 Shared-Memory Layout and Alignment

To maximize AXI bus efficiency and minimize address calculation overhead in hardware, the shared memory follows a strictly contiguous and aligned layout. All buffers are aligned to a **64-byte boundary**, matching typical AXI burst sizes and cache line requirements.

The memory region starts at a configurable base address, `VIBE_FPGA_SHARED_BASE`, which defaults to `0xC4000000`. The data structures are packed in the following sequence:

1. **Current Frame Buffer:** The raw input image pixels currently being processed.
2. **History Samples:** The temporal background model consisting of multiple samples per pixel.
3. **Output Words:** The generated foreground mask or classification results.
4. **Control Block:** The metadata structure containing algorithmic thresholds (e.g., R , N_{min}) and status flags.

[Current Frame (64B Aligned)] → [History (64B Aligned)] → [Output (64B Aligned)] → [Control Block]

Figure 3: Shared Memory Sequential Layout Contract

9.3 Execution Flow and Invariants

Shared-memory layout stability is a strict invariant. By placing the **Control Block** at the end of the region, the software can update parameters or poll status without interrupting the high-bandwidth streaming of frame and history data.

Before triggering the `START` bit in the `CONTROL` register, the host software must populate the `CONTROL_BLOCK_ADDR` with the absolute physical address of the descriptor. The hardware DMA engine uses this pointer to fetch algorithmic constants before commencing the pixel-processing pipeline. Any violation of the 64-byte alignment during address calculation will trigger a fault visible in the `DEBUG_ERROR_CODE` register.

10 M2 Hardware Architecture and Matching Logic

The custom M2 hardware module is designed as a high-efficiency transaction executor optimized for the BeagleV-Fire's SoC fabric. It operates across dual logical planes to decouple system control from heavy data movement.

10.1 System Interconnect: Dual-Plane Architecture

The M2 module leverages two distinct bus protocols to balance latency and throughput:

- **APB Control Plane:** A low-latency peripheral bus used for task initiation, status observation, and access to internal debug registers. This plane handles the "handshake" between the CPU and the hardware.
- **AXI Data Plane:** A high-bandwidth master interface responsible for autonomous data movement. It handles the retrieval of control blocks, the streaming of interleaved pixel data, and the burst-writeback of comparison results.

10.2 Data Alignment and AXI Bandwidth Optimization

In the original software implementation, pixels are stored in a compact 24-bit (3-channel $\times$ 8-bit) format. However, the AXI4 bus on the RISC-V fabric is optimized for 64-bit transactions. Accessing 24-bit unaligned data would result in significant bus overhead and inefficient burst cycles.

To maximize throughput, the hardware interface implements a **64-bit Word Expansion Strategy**:

- Each pixel-history sample pair is padded and aligned to a 64-bit boundary in memory.
- This alignment ensures that every AXI read operation fetches a complete, valid data package without requiring complex byte-shuffling logic at the hardware front-end.
- This trade-off—sacrificing a small amount of memory density for a massive gain in memory bus efficiency—allows the M2 engine to saturate the AXI interconnect capacity.

10.3 Fixed-Point Computation and Pre-calculated Thresholds

To minimize logic resource consumption (LUTs/DSPs) and ensure deterministic single-cycle execution, the hardware implementation avoids all floating-point operations.

10.3.1 SAD Matching Rule

The core matching rule utilizes the Sum of Absolute Differences (SAD) across the color channels. For a current pixel c and a history sample h , the match is determined by:

$$|B_c - B_h| + |G_c - G_h| + |R_c - R_h| \leq T_{hw} \quad (2)$$

10.3.2 Pre-calculation Strategy

In the baseline C3R algorithm, the threshold is typically expressed as $4.5 \times T$. To avoid performing a floating-point multiplication (or a complex fixed-point approximation) within the hardware pipeline, the following optimization is applied:

- **Offline Scaling:** The constant factor 4.5 is folded into the threshold during the software initialization phase.
- **Integer Comparison:** The hardware receives a pre-calculated integer threshold $T_{hw} = \lfloor 4.5 \times T \rfloor$.
- **Logic Efficiency:** This reduces the matching logic to simple subtraction, absolute value (MUX-based), and addition, ensuring the entire SAD operation can be completed within a single clock cycle at high frequencies.

10.4 Evidence Word Generation

Hardware iterates through all history samples for each pixel and constructs a **Comparison Word** (Evidence Bitmap). For each sample i in the set N :

- If the SAD test passes, the i -th bit of the word is set to 1.
- If it fails, the bit is set to 0.

This bitmask is then written back to the shared memory via the AXI master, shifting the final "counting and policy decision" (Is $\text{bitcount} \geq \text{min?}$) to the software, thereby maintaining a flexible classification threshold.

11 State Machine, CDC, and Error Policy

At state-machine level, the hardware flow can be summarized as:

```
idle wait -> control-block read/validate -> pixel/sample loops ->
writeback -> done or error
```

On protocol errors, parameter overflow, or bus response failures, hardware enters an error path and writes an error code. Error codes should not be treated as log-only artifacts; software should incorporate them into control policy (for example, degrade, retry, or frame-drop protection).

For clock-domain crossing between APB and AXI, critical events (start, done, error) are passed via synchronizer chains and toggle-based mechanisms, rather than single-cycle pulse transfer.

12 Performance and Validation Methodology

The dominant bottleneck is memory bandwidth rather than arithmetic comparison. Traffic scales approximately linearly with pixel count and history size. Therefore, the project follows a staged strategy:

1. stabilize interface semantics,
2. verify functional correctness,
3. then optimize through burst transfer, on-chip buffering, and batch processing.

For debugging, the most effective baseline is a small hand-computable closed loop: use tiny images and controlled histories, manually derive comparison words, compare bit-by-bit with hardware outputs, and verify software decisions.

13 Gateway Build and Flashing Workflow

Beyond algorithm and interface design, a repeatable production-grade workflow is required. Based on repository scripts and public documentation, the recommended flow is as follows.

Step 1: Host Environment Preparation

Use Ubuntu or WSL Linux. Install Python3, git, device-tree-compiler, and Python packages GitPython and PyYAML. Ensure Libero, MSS Configurator, and SoftConsole are available with valid licensing.

Step 2: Environment Variable Verification

At minimum verify SC_INSTALL_DIR, FPGENPROG, LIBERO_INSTALL_DIR, and LM_LICENSE_FILE. Ensure PATH can find libero, pfsoc_mss, riscv-unknown-elf-gcc, and dtc.

Step 3: Build Configuration Selection

Use either `build-options/default.yaml` or `build-options/m2-custom-axi.yaml`. This configuration controls gateway options and HSS source version.

Step 4: Bitstream Build

```
1 python3 build-bitstream.py ./build-options/m2-custom-axi.yaml
2 python3 build-bitstream.py ./build-options/m2-custom-axi.yaml ./boards/mpfs-
  beaglev-fire.yaml
```

Key artifacts are generated under `bitstream/LinuxProgramming`, especially `mpfs_bitstream.spi` and `mpfs_dtbo.spi`.

Step 5: Copy Artifacts to Board

```
1 scp -r bitstream/LinuxProgramming debian@192.168.7.2:/opt/gateway/custom-m2/
```

Step 6: Trigger Flashing

```
1 sudo /usr/share/microchip/gateway/change-gateway.sh /opt/gateway/custom-m2
```

Step 7: Post-Reboot Verification

Verify control-plane, data-plane, and algorithm-output correctness: APB register accessibility, AXI read/write path, and one-frame minimal BGS output consistency.

Step 8: Rollback and Safety

Keep at least one validated rollback pair of `mpfs_bitstream.spi` and `mpfs_dtbo.spi`. Because flashing updates MTD regions, perform eMMC backup before critical tests and ensure stable power during update.

14 Evidence from Source Structure and Commit History

The architecture can be reconstructed from code structure and history:

- Software/protocol side centralizes control block definitions, status bits, layout routines, and decision functions in a shared header, with naming aligned to hardware read order.
- Hardware side separates APB register plane and AXI data plane, and integrates CDC synchronization, control-block reads, validation, traversal, writeback, and error handling in one module.

- Commit evolution indicates two-way convergence: runnable hardware backbone first, then control-plane definition completion on software side, and final semantic unification.

15 System Collaboration and Hardware Logic

To further illustrate the interaction between the software-defined ViBe logic and the M2 hardware executor, this section provides the formalized execution flow and the internal state machine transitions.

15.1 System-Level Collaboration Flow

The following diagram describes the full transaction cycle, from memory preparation to the final foreground decision.

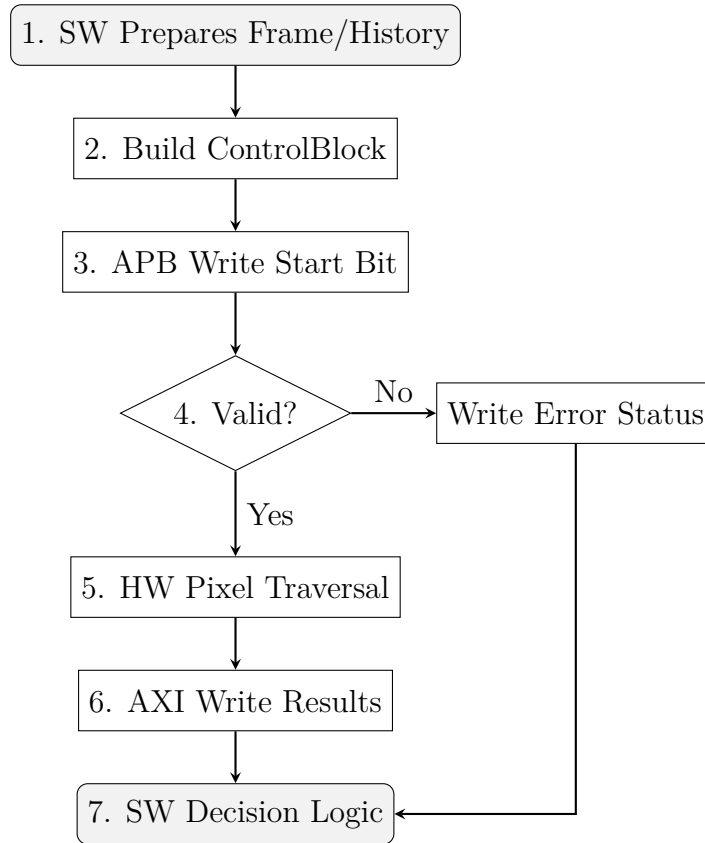


Figure 4: Cross-plane interaction flow between CPU and M2 Hardware.

15.2 Hardware Finite State Machine (FSM) Abstraction

The internal logic of the M2 core is governed by a deterministic state machine, ensuring precise AXI burst timing and error handling.

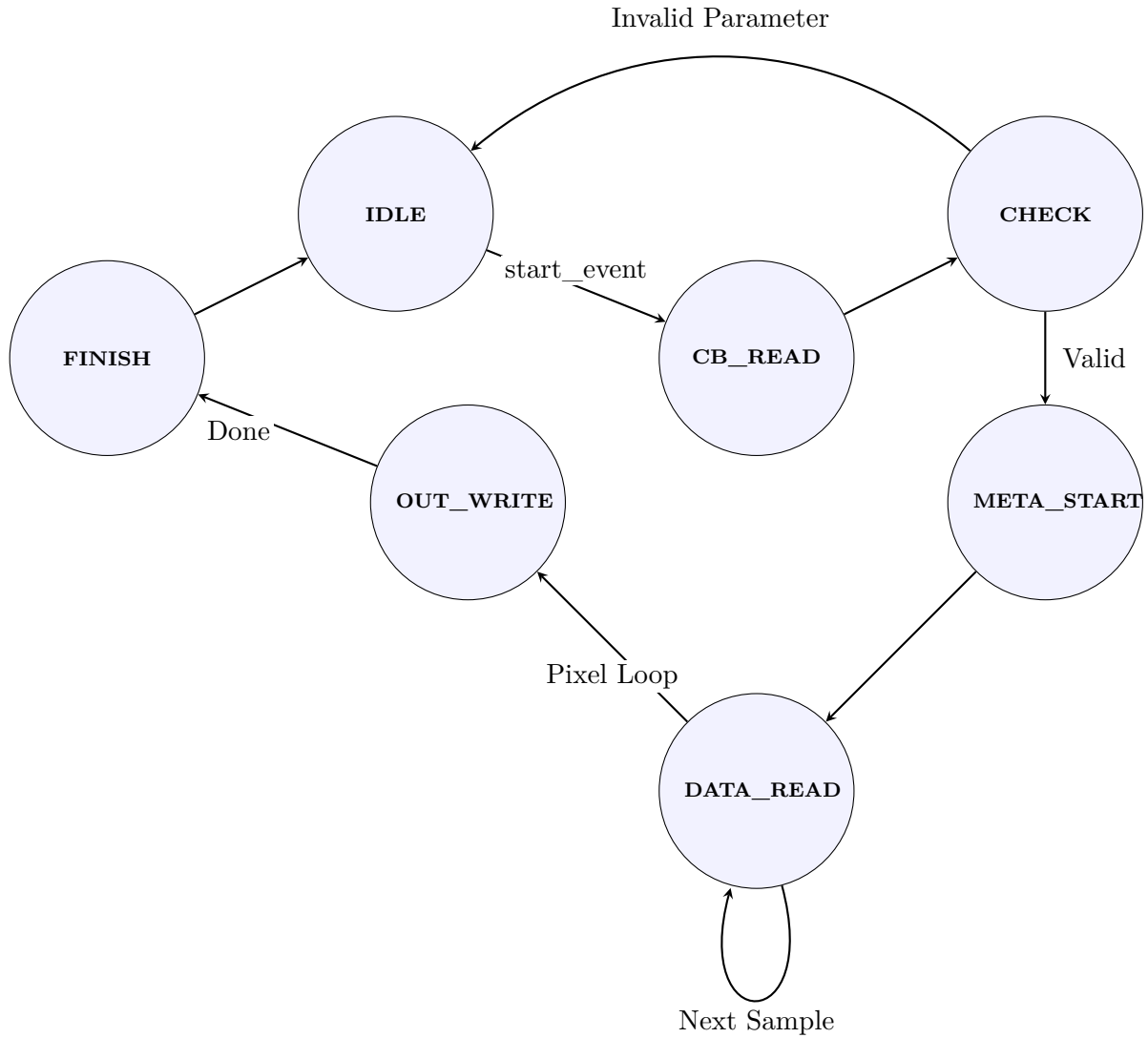


Figure 5: M2 internal FSM for pixel matching and data movement.

16 Conclusion

The project demonstrates not only successful migration and validation of Vibe on a constrained platform, but also construction of a long-term evolvable software-hardware interface system. Its central engineering value is the decomposition of complexity into three independently evolvable objects: task description, hardware execution, and software interpretation. With stable interface semantics and strict versioning discipline, the system can continue to evolve while maintaining compatibility.

17 Summary and Semester Reflection

Looking back at this semester, this project served as a bridge between my academic knowledge and real-world engineering challenges. Starting with the initial “bring-up” of the development board in March, I faced numerous practical hurdles, such as OS re-flashing

and network adaptation, which are rarely covered in standard textbooks. Overcoming these obstacles provided me with invaluable hands-on experience with Linux-based embedded environments and toolchain management.

In the second half of the semester, focusing on the ViBe algorithm and M2 accelerator integration allowed me to explore the intricacies of hardware-software partitioning. I realized that successful co-design is not just about performance optimization, but about creating stable, well-documented interfaces that allow different system components to evolve independently.

This Independent Study has significantly improved my skills in C/C++ cross-compilation, GStreamer-based streaming, and hardware FSM design. More importantly, it has instilled in me a rigorous mindset toward documentation and system observability, which I will carry forward into my future research and professional career.